Guia rápido · Módulo 08: Banco de Dados com MongoDB e Mongoose

Resumo para consulta rápida. A explicação completa está nos slides e nos arquivos desta pasta.

Conceitos

Relacional (SQL)	MongoDB
tabela	coleção
linha	documento (JSON)
colunas fixas	campos flexíveis

Embedding (documento dentro do outro): dados que sempre andam juntos. **Referência** (guarda o `_id`): dados que crescem ou são usados sozinhos, como Flashcard → Baralho.

Atlas e .env

1. Crie o cluster (plano free), o **usuário do banco** e libere o IP em **Network Access**.
2. Copie a connection string e coloque no `.env`:

```
MONGO_URI=mongodb+srv://usuario:senha@cluster0.xxxx.mongodb.net/flashcards
```

3. `.env` no `.gitignore`; um `.env.example` sem valores vai para o Git.

Conexão

```
import mongoose from "mongoose"

async function conectarBanco() {
  try {
    await mongoose.connect(process.env.MONGO_URI)
    console.log("Conectado ao MongoDB")
  } catch (error) {
    console.error("Erro ao conectar:", error.message)
    process.exit(1)
  }
}
export default conectarBanco
```

Schema e model

```
const FlashcardSchema = new mongoose.Schema({
  pergunta: { type: String, required: [true, "A pergunta é obrigatória."], trim: true },
  resposta: { type: String, required: true },
  baralho: { type: mongoose.Schema.Types.ObjectId, ref: "Baralho", required: true },
}, { timestamps: true })          // createdAt e updatedAt

export default mongoose.model("Flashcard", FlashcardSchema)
```

Operações (todas com await)

```
await Flashcard.create(dados)
await Flashcard.find()                        // todos
await Flashcard.find({ baralho: id })         // com filtro
await Flashcard.findById(id)                  // um (ou null)
await Flashcard.findById(id).populate("baralho")
await Flashcard.findByIdAndUpdate(id, dados, { new: true, runValidators: true })
await Flashcard.findByIdAndDelete(id)
```

Rota com banco

```
app.get("/flashcards/:id", async (req, res) => {
  try {
    const card = await Flashcard.findById(req.params.id)
    if (!card) return res.status(404).json({ message: "Não encontrado." })
    return res.json(card)
  } catch (error) {
    return res.status(400).json({ message: "ID inválido." }) // CastError
  }
})
```

Atualização parcial

- `findByIdAndUpdate` usa `$set` implícito: só os campos enviados mudam.
- **Nunca** passe o `req.body` inteiro: monte uma lista de campos permitidos.
- `$unset` remove o campo do documento; gravar `null` mantém o campo, vazio.

Soft delete

```
// excluir = marcar a data
await Flashcard.findByIdAndUpdate(id, { excluidoEm: new Date() })
// listar só os ativos
await Flashcard.find({ excluidoEm: null })
// buscar um ativo (o findById não sabe do soft delete!)
await Flashcard.findOne({ _id: id, excluidoEm: null })
// restaurar
await Flashcard.findByIdAndUpdate(id, { excluidoEm: null })
```

LGPD: se a pessoa pedir a exclusão dos dados pessoais, o soft delete não basta.

Armadilhas

- Sem `await`: vem uma consulta pendente, não os dados.
- `.env` não carregado → *MONGO_URI não configurada* / `undefined`.
- Timeout na conexão: IP não liberado no Atlas.
- Senha com `@` ou `#` na URI: codifique (`encodeURIComponent`).
- `ValidationError` (campo obrigatório) e `CastError` (id malformado) → responda 400.